BRAUDE College of Engineering, Karmiel

Software Engineering Department

Capstone Project - Phase B

Atmospheric Simulation

A 3D Particle-Based Planetary Atmosphere Model

Team 26-1-R-13 | Yasser Sadi & Diana Hujerat | Advisor: Dr. Zakharia Frenkel

SCAN ME

GitHub Repository QR

Live Demonstration

Background & Motivation

The Problem: Atmospheric circulation is a three-dimensional process shaped by gravity, heat, rotation, and moisture — yet most models still solve continuous fluid equations on a grid.

Our Approach: A Lagrangian alternative — thousands of discrete air parcels interacting inside a spherical shell, rather than a fixed grid.

Goal: An educational, research-oriented framework that simulates parcel transport and generates reusable diagnostics from saved simulation data.

Project Objectives

- ✧ Build a 3D parcel simulator inside a spherical shell.
- ✧ Keep parcel motion stable using gravity, repulsion, damping, and boundary reflection.
- ✧ Model heating that changes from equator to pole and with altitude.
- ✧ Look at spherical velocities, meridional circulation, and streamfunction-like fields.
- ✧ Add humidity transport, evaporation, condensation, latent heating, and water tracking.
- ✧ Automate the build, run, output check, and visualization steps.

Key Requirements

Functional

- ✧ Simulation output for 10,000 parcels.
- ✧ CSV files for particles, grids, circulation, streamfunction, and moisture.
- ✧ An interactive dashboard, plus report figures, animation, and export files.

Quality attributes

- ✧ A clean-run pipeline that gives the same results each time.
- ✧ Simulation, analysis, I/O, and visualization kept separate so the code is easier to maintain.
- ✧ Stable, finite numbers everywhere — nothing breaks or blows up.
- ✧ Every claim backed up by saved diagnostic data.

Engineering / Research Process

Stage 1	**Stable shell**	Particle bounds, radial density, energy diagnostics
Stage 2	**Thermal forcing**	Temperature zones, altitude profiles, radial velocity
Stage 3	**Rotation + circulation**	vθ, vϕ, accumulation, Ψ output
Stage 4	**Moisture cycle**	qp, evaporation, condensation, latent heating, water balance
Visual-ization	**Result delivery**	Dashboard, HTML, PNG/GIF, VTK, report figures

Technologies & Tools

Core	C++17, CMake, standard library
Numerics	Velocity Verlet, 3D spatial hashing, coarse aggregation
Visualization	Python, Pandas, NumPy, Matplotlib, Plotly, ImageIO
Environment	Windows, WSL Ubuntu, VS Code / Cursor
Versioning	Git, GitHub
Exports	CSV, HTML, PNG, GIF, VTK / ParaView

Proposed Solution: Layered Architecture

1. C++ Simulation Engine
10,000 parcel states | forces | Velocity Verlet | thermal forcing | rotation | moisture | diagnostics

↓

2. Structured Output Contract
Particle snapshots | spherical fields | grid diagnostics | step-indexed CSV outputs

↓

3. Python Post-Processing
Pandas/NumPy loaders | Matplotlib/Plotly figures | animation | VTK | manifest generation

↓

4. Result Delivery
Local dashboard | interactive 3D viewers | GIF/PNG figures | CSV/HTML/VTK export

Development and Execution Flow

1. **Configure & build** — CMake Release and C++ compilation
2. **Prepare clean output** — Refresh simulation and website folders
3. **Run simulation** — Execute the 10,000-parcel production model
4. **Verify CSV files** — Confirm current-run output exists
5. **Generate outputs** — Create figures, 3D HTML, animation, VTK, manifest
6. **Inspect dashboard** — Open visualization_output through localhost

Main Algorithms & Technical Contribution

Particle containment
Problem: Parcels could drift outside the shell.
Method: Push them back radially and flip the radial part of their velocity at R and R+H.
Benefit: Parcels stay inside the shell, and their sideways motion isn't affected.

Local interactions
Problem: Checking repulsion between every pair of parcels costs too much.
Method: Use a grid with σ-sized cells and only check the 26 cells around each one.
Benefit: Cuts the work down a lot, since each parcel only checks nearby cells.

Time integration
Problem: The particle motion needs a stable way to step forward in time.
Method: Velocity Verlet with dt = 0.01.
Benefit: A solid, second-order method that fits this kind of MD-style simulation well.

Circulation diagnostic
Problem: A single snapshot in time is too noisy to read.
Method: Build up a 36×20 latitude-altitude grid over a long stretch of time and use it to get a Ψ value.
Benefit: Shows clear, organized circulation patterns in the late-time data.

Testing & Evaluation

Area	Method	Acceptance evidence
Build/run	Release build + 100,000-step execution	Fresh CSV files
Geometry	Radius-boundary checks	Particles stay inside shell
Rotation	Code design + runtime transition	Single inertial-frame Ω×r imprint
Circulation	Finite vθ, mass-flux proxy, and Ψ files	Finite reproducible fields
Moisture	qp bounds, source/sink logs, water balance	Finite bounded accounting
Visualization	Dashboard regeneration and browser checks	Assets open correctly

Engineering Challenges & Solutions

PC Particle Containment
Challenge: Stop 10,000 moving parcels from escaping the spherical shell.
Solution: Fix their radial position and flip only the radial part of their velocity at the inner and outer edges.
Impact: Every parcel stays inside, and its sideways motion is untouched.

N² Computational Cost
Challenge: Challenge: Checking every pair of parcels directly would take O(N²) work.
Solution: Use a 3D grid and only check a parcel's own cell plus its 26 neighbors.
Impact: Cuts the work way down and makes a 10,000-parcel run actually doable.

S Initialization Instability
Challenge: Starting with random positions and speeds caused big, messy jumps early on.
Solution: Add strong damping during the early settling-in phase, then turn it off for good.
Impact: The shell settles down without flattening out the real atmospheric motion later.

Ω Rotation Modeling
Challenge: Add planetary rotation without counting any force twice or adding a fake force by mistake.
Solution: Add the rotation speed Ω × r exactly once, right at the start of the rotation phase.
Impact: Gives correct planetary rotation, with no double-counting and no extra made-up force.

Ψ Noisy Circulation Diagnostics
Challenge: Looking at particle speeds at a single moment was too noisy to make sense of.
Solution: Add up velocity and mass-flow data over a long stretch of time on a 36×20 grid before working out Ψ.
Impact: Gives much steadier, easier-to-read circulation results.

3D Visualization Performance & Readability
Challenge: Showing every particle slowed the browser down, and fixed color ranges sometimes hid real patterns.
Solution: Only show a smaller sample of particles, and pick color ranges based on the actual data, centered on zero where it makes sense.
Impact: Keeps all the real simulation data while making the 3D views and heatmaps fast and easy to read.

Project Metrics & Achievement

Metric	Target	Observed	Status
Production run	100,000 steps	Completed	Achieved
Parcel population	10,000 parcels	Implemented	Achieved
Clean pipeline	One command to dashboard	run_full_pipeline.sh	Achieved
Thermal diagnostics	Latitude + altitude outputs	Generated	Achieved
Rotation handling	Ω×r exactly once	Implemented	Achieved
Circulation diagnostic	Finite vθ and Ψ	Generated and visualized	Achieved
Moisture outputs	Evap/cond/latent heat	Generated and logged	Achieved

Engineering Highlights & Next Steps

Engineering highlights

- ✧ Stable containment → radial projection and reflection at both shell boundaries.
- ✧ Efficient force computation → local 3D grid neighbor search.
- ✧ Robust circulation diagnostics → late-time latitude-altitude accumulation before Ψ calculation.
- ✧ Clean reproducibility → automatic cleanup before each production run.

Conclusions

- ✧ A full parcel-based atmosphere simulation was built and written up.
- ✧ Its biggest strength is the full pipeline — from the C++ simulation all the way to the dashboard.
- ✧ The saved data shows heating differences, rotation effects, organized circulation, and an active water cycle.
- ✧ The saved outputs, repeatable figures, and interactive dashboard all back up these results well.

Next development opportunities

- ✧ Add a setting to control when moisture turns on, for longer test runs.
- ✧ Make the ParaView export tools better.
- ✧ Add more automatic tests to check that CSV files are fresh and correctly formatted.
- ✧ Add more options for density weighting, streamfunction analysis, running many parameter sets, and saving/restarting runs.

Molecular Dynamics–Inspired Method

The atmosphere is represented by **10,000 Lagrangian air parcels** — macroscopic atmospheric parcels, **not real air molecules.**

Core equations

Newton's second law $m_i a_i = F_i$

Net-force model

$$F_i = Fg_{,i} + \Sigma_{j \in N(i)} Frep_{,ij} + Fdamp_{,i}$$

Velocity Verlet integration

Half-step velocity $v_i^{n+1/2} = v_i^n + (\Delta t / 2m_i) F_i^n$

Position update $r_i^{n+1} = r_i^n + \Delta t \cdot v_i^{n+1/2}$

Final velocity $v_i^{n+1} = v_i^{n+1/2} + (\Delta t / 2m_i) F_i^{n+1}$

Performance O(N²) avoided
Current cell + 26 neighbors only; direct O(N²) avoided.

Method flow

1. Parcel State
2. Local Force Calculation
3. Velocity Verlet
4. Boundary Correction

Main Results: Structure and Diagnostics

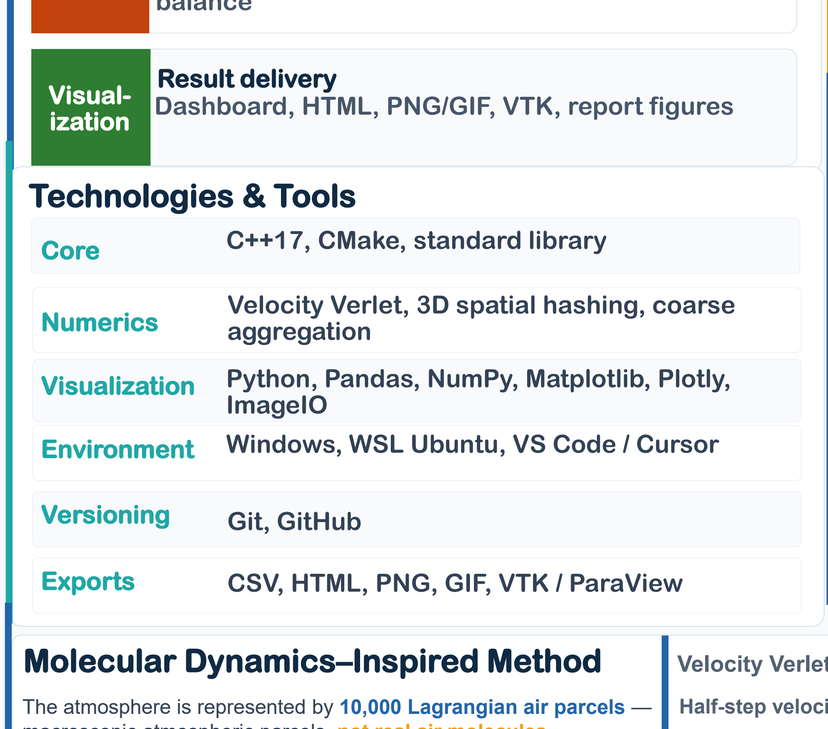

Figure 1. Streamfunction-like Ψ at step 100,000. Finite opposite-signed regions show organized meridional transport in the diagnostic field.

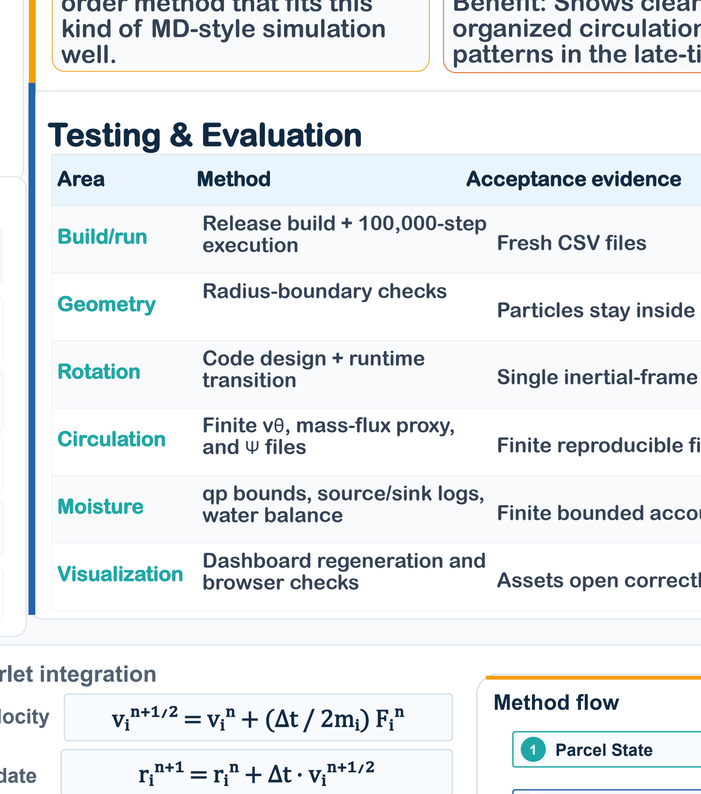

Figure 2. Particle shell preview at step 40,000.

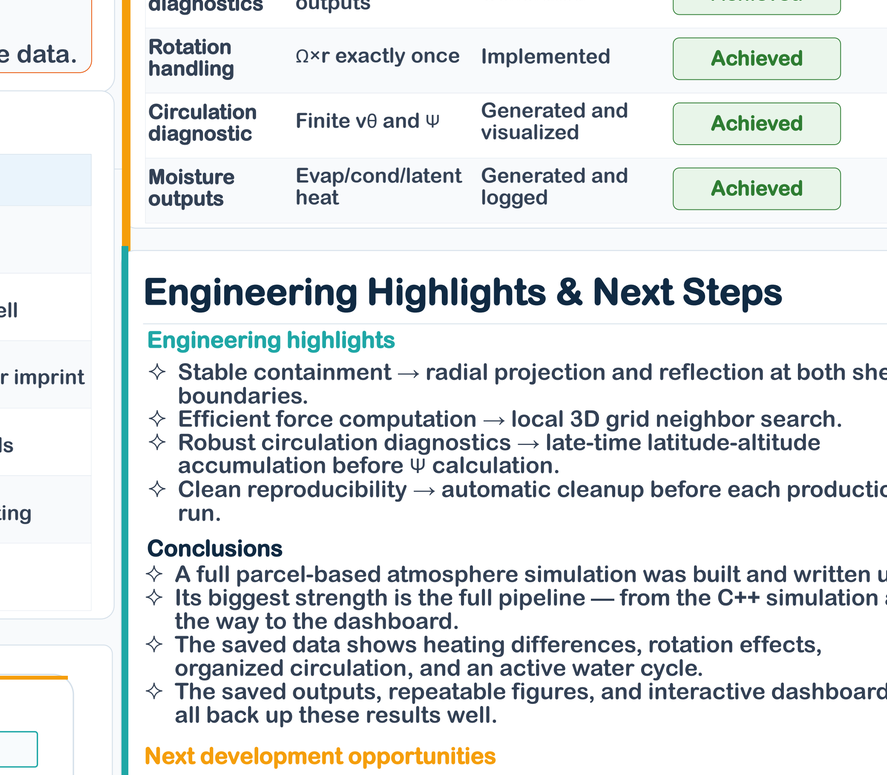

Figure 3. Longitude-averaged meridional velocity vϑ across latitude and altitude.